

Task 1

Figure out the data types/ dataset types

1. Synthetic :- as the name implies this build out synthetic datasets to eliminate input and pipeline bottlenecks
2. Grain :- Recommended choice by the devs as it's most importantly deterministic, so in case you have a failure mid-training and have to restart, you won't lose the progress, as it can continue from the same point, as the data doesn't change
3. HuggingFace pipeline :- It allows you to directly acquire data from a HF hub or storage bucket
4. TFDS- this is to download/use datasets from Google's AllenAI C4 dataset, but on non google cloud environments you will have to pay for it

Dataset Name	Advantage	Disadvantage
Synthetic	No network or data loading pipeline bottlenecks	• No real data loss metric • No padding for tokens (for real data you get padding) • Can't catch bugs in tokenizer or data distribution
Grain	Determinism allows us to save time and money in case of pipeline failures	• Most complex to set up • Determinism is only fully available with ArrayRecord method, which requires the data to go through a conversion pipeline for a custom format
Hugging Face	Access to HF datasets directly without the need to download	• Network throttling issues • Which might require downloading the dataset • In case of pipeline failures, no random access or checkpoints, so you restart
TFDS	Access to super useful AllenAI C4 dataset	• Requires you to either use and pay for GCS bucket or for a non-Google Cloud environment pay per gb download of the dataset • No checkpoint / random access

TASK 2

General info; a lot of info has been taken from the calculator and other parts of wiki of Maxtext.

While working on the first problem, despite most efforts I was unable to get the model running at 0.6B parameters using the CPU.

I used the calculator provided to figure out the MLP, and embedding dimensions; I had almost 6-7 runs for the scaled version where it would have around 91 to 750 Mib extra then what was allotted. The 0.6B version on the gpu ran well.

For the TPU as shown in the table

Model & Run Configuration	Backend	Sequence Length	Run Status / Outcome	Memory Usage / Pool	Compute Throughput (TFLOP/s/device)	Token Throughput (Tokens/s/device)	Parameter Count	Key Architectural / Config Settings Used
Qwen3 Dense <i>(Baseline Run)</i>	CPU	512	Failed (OOM / OS SIGKILL)	~12.2 GB (Estimated required)	—	—	0.596B	hardware=cpu Tied embeddings (logits_via_embedding=True)
Qwen3 Dense <i>(Baseline Run)</i>	GPU (T4)	2,048	Completed	6.66 / 10.92 GB (61.0% VRAM)	~2.03	~475	0.596B	attention=cudnn_flash_te scan_layers=False use_jota_embed=True packing=False

								(Unfused fallback kernel active)
Qwen3 Dense (Baseline Run)	TPU (v5e)	2,048	Completed	6.69 / 15.75 GB (42.5% HBM)	~71.2	~16,630	0.596B	attention=autoselected (Simplest configuration path)
Qwen3 Dense (Scaled-Up Run)	GPU (T4)	256	Completed	8.03 / 10.92 GB (73.5% VRAM)	~2.00	~454	0.719B	Scaled via base_mlp_dim (3072 → 4500) base_emb_dim held at 1024
Qwen3 Dense (Scaled-Up Run)	TPU (v5e)	—	Failed (OOM Wall)	Exceeded 15.75 GB	—	—	~1.09B (Target)	Attempted scaling via emb_dim=1536 / mlp_dim=4608 Hit tied backprop memory wall

Task 3

I scaled the model down initially by reducing the number of experts first, then reduced the embedding to the Qwen 3 levels. Using the calculator, it was clear that this was not enough so I went through the documentation to understand what can be reduced to get to 1B. I then reduced num of query heads and max target length; I also reduced LoRA ranking after that the changes are mentioned clearly in the following table

Parameter	Reduction reason
-----------	------------------

max_target_length	Memory reduction as memory for attention scales quadratically
Batch per device	Reduced to be able to use on free TPU
num_kv_heads	Changed to match other features
num_query_heads	Read that reducing this will reduce low rank projections
Emb_dim	Reduced to reduce output layer reducing total internal layers as well
Num of experts	Each expert multiplies moe_mlp_dim with um_moe_layers

Model & Run Configuration	Backend	Sequence Length	Run Status / Outcome	Memory Usage / Pool	Compute Throughput (TFLOP/s/device)	Token Throughput (Tokenss/device)	Parameter Count	Key Architectural / Config Settings Used
DeepSeek MoE <i>(Initial Attempt)</i>	TPU (v5e)	512	Failed (OOM before step 1)	17.85 GB needed / 15.75 GB available	—	—	—	128 Query/KV Heads retained from 671B base config

DeepSee k MoE <i>(Successful Scaling)</i>	TPU (v5e)	256	Comp leted 	11.29 / 15.75 GB (71.7% HBM)	—	—	1.007B	Untied embeddings (<code>logits_via_embedding=False</code>) 32 Heads (<code>num_query_heads == num_kv_heads</code> via MLA restriction) 12 active layers (3 dense + 12 MoE / 13 total experts) Loss collapsed 12.3 → 0.001 (Synthetic data footprint)
DeepSee k MoE <i>(Projected Run)</i>	TPU (v5e)	256	Valid ated (Math only)	~10.9 GB (Projec ted)	—	—	~0.969B	Reduced to 11 experts to fall strictly below <1B parameter ceiling
DeepSee k MoE <i>(All Configura tions)</i>	GPU (T4)	—	Not Comp leted (Skipp ed / OOM)	Pool too small (10.92 GB < 11.29 GB require d)	—	—	—	Architecture size exceeds hardware HBM pool limits; MLA requires <code>attention=dot_product</code> fallback

DeepSee k MoE	CPU	—	Not Comp leted (Skipp ed / OOM)	Free-ti er RAM insuffici ent	—	—	—	Skipped due to baseline dense 0.6B failure on identical memory constraints
(All Configura tions)								

BEenchmarking has two main things MFU and throughput [link](#)